BTS CIEL · PROGRAMMATION C++ · TRAVAUX PRATIQUES

TP06 — Calculateur de réseau IPv4

Prérequis : chapitres 1 à 6

Contexte professionnel

Lors du déploiement d'un réseau d'entreprise, le technicien doit vérifier rapidement le plan d'adressage : à partir d'une adresse et d'un préfixe CIDR, il veut connaître le masque, l'adresse réseau, l'adresse de diffusion et la plage des hôtes utilisables. Les calculateurs en ligne ne sont pas toujours accessibles sur site : on vous demande un outil en ligne de commande.

Objectifs

- Représenter une adresse IPv4 dans un `std::uint32_t`.
- Assembler et extraire des octets avec les décalages et les masques.
- Construire un masque à partir d'un préfixe en évitant tout décalage interdit.
- Calculer adresse réseau, diffusion et plage d'hôtes, cas limites `/0`, `/31` et `/32` compris.
- Rendre les saisies robustes.

Compétences mobilisées

Compétence	Mise en œuvre dans ce TP
C04 — Analyser un système informatique	Analyser le plan d'adressage, calculer à la main en binaire
C05 — Concevoir un système informatique	Découper le calculateur en fonctions spécifiées
C06 — Valider un système informatique	Valider le calculateur sur les cas ordinaires et limites
C08 — Coder	Écrire les fonctions de calcul, d'affichage et de saisie

Cahier des charges

Le programme demande les quatre octets d'une adresse (chacun de 0 à 255), puis un préfixe (de 0 à 32). Toute saisie non numérique ou hors domaine est signalée puis **redemandée**. Il affiche ensuite :

Ligne	Calcul
Adresse IP	L'adresse saisie
Masque	Bits à 1 sur les `prefixe` bits de poids fort
Adresse réseau	Adresse `&` masque
Broadcast	Adresse réseau `|` (`~` masque)
Hôtes	Nombre d'adresses d'hôtes utilisables
Premier hôte, dernier hôte	Bornes de la plage d'hôtes

Nombre d'hôtes et plage selon le préfixe :

Préfixe	Nombre d'hôtes	Premier hôte	Dernier hôte
0 à 30	2^(32 − préfixe) − 2	réseau + 1	broadcast − 1
31 (liaison point à point)	2	adresse réseau	broadcast
32 (adresse d'un hôte unique)	1	l'adresse elle-même	l'adresse elle-même

Fonctions imposées

Prototype	Rôle
`bool lireEntier(int minimum, int maximum, int& valeur)`	Saisie robuste ; renvoie `false` si l'entrée est fermée
`std::uint32_t assemblerAdresse(std::uint32_t o1, std::uint32_t o2, std::uint32_t o3, std::uint32_t o4)`	Assemble les quatre octets
`bool construireMasque(int prefixe, std::uint32_t& masque)`	Construit le masque ; renvoie `false` (et ne modifie pas `masque`) si le préfixe n'est pas entre 0 et 32
`void afficherAdresse(std::uint32_t adresse)`	Affiche en notation pointée
`std::uint64_t nombreHotes(int prefixe)`	Nombre d'hôtes (`std::uint64_t` : entier non signé de 64 bits, de `<cstdint>`, car 2^{32} ne tient pas dans 32 bits)

Travail demandé

Partie 1 — Analyse (à la main, en binaire)

1. Pour `192.168.10.77/26` : écrivez l'adresse et le masque en binaire, puis calculez l'adresse réseau, le broadcast, le nombre d'hôtes et la plage.
2. Pourquoi `0xFFFFFFFFu << (32 - prefixe)` ne convient-il pas pour `/0` ? Que proposez-vous ?
3. Pourquoi faut-il décaler un `std::uint32_t` (et non un `int`) pour assembler l'adresse ?

Partie 2 — Calculs

4. Écrivez `assemblerAdresse`, `construireMasque`, `afficherAdresse` et `nombreHotes`.
5. Écrivez dans `main()` des tests de `construireMasque` pour les préfixes 0, 8, 24, 26, 32 (masques attendus `0`, `FF000000`, `FFFFFF00`, `FFFFFFC0`, `FFFFFFFF`), et pour les préfixes invalides -1 et 33 (retour `false`, masque inchangé).

Partie 3 — Programme complet

6. Écrivez `lireEntier`, puis le programme complet : saisies, calculs, affichage au format ci-dessous.

Partie 4 — Validation

7. Déroulez tous les tests du tableau.

Pour aller plus loin

8. Demandez une seconde adresse et indiquez si elle appartient au même sous-réseau que la première.

Contraintes techniques

- Compilation avec `g++ -std=c++17 -Wall -Wextra`, sans erreur ni avertissement.
- Aucun décalage de 32 bits ou plus, aucun décalage d'entier signé.
- Les calculs (masque, réseau, broadcast) n'affichent rien ; seule `afficherAdresse` affiche une adresse.
- Constantes pour les bornes des saisies.

Tests à effectuer

N°	Saisie	Masque	Réseau	Broadcast	Hôtes	Premier	Dernier
1	`192 168 10 77 /26`	255.255.255.192	192.168.10.64	192.168.10.127	62	192.168.10.65	192.168.10.126
2	`10 0 0 1 /8`	255.0.0.0	10.0.0.0	10.255.255.255	16777214	10.0.0.1	10.255.255.254
3	`192 168 1 130 /25`	255.255.255.128	192.168.1.128	192.168.1.255	126	192.168.1.129	192.168.1.254
4	`192 168 1 1 /31`	255.255.255.254	192.168.1.0	192.168.1.1	2	192.168.1.0	192.168.1.1
5	`172 16 5 4 /32`	255.255.255.255	172.16.5.4	172.16.5.4	1	172.16.5.4	172.16.5.4
6	`8 8 8 8 /0`	0.0.0.0	0.0.0.0	255.255.255.255	4294967294	0.0.0.1	255.255.255.254
7	`300`, puis `abc`, puis `192` ; …	Les deux premières saisies sont redemandées					
8	Préfixe `33`, puis `24`	La première saisie est redemandée					

Affichage attendu pour le test 1 :

```
Adresse IP      : 192.168.10.77
Masque          : 255.255.255.192
Adresse reseau  : 192.168.10.64
Broadcast       : 192.168.10.127
Hotes           : 62
Premier hote    : 192.168.10.65
Dernier hote    : 192.168.10.126
```

Livrables

- Le fichier `calculateur_ipv4.cpp`.
- La feuille d'analyse (partie 1) et le compte rendu des tests.

Critères d'évaluation

Critère	Indicateurs de réussite	Points
Analyse	Calculs binaires exacts ; cas `/0` et décalage d'un non signé justifiés	4
Assemblage et affichage	Octets placés et extraits correctement, notation pointée	3
Masque	`construireMasque` exact pour 0 à 32, préfixes invalides refusés, tests présents	4
Réseau, broadcast, hôtes	Résultats exacts, y compris `/0`, `/31` et `/32`	4
Saisies robustes	Non numérique et hors domaine redemandés ; fin d'entrée gérée	3
Qualité	Fonctions imposées respectées, aucun affichage dans les calculs, lisibilité	2
Total		**20**